

Planejamento de testes - ServeRest

Planejamento de testes - ServeRest

1. Visão Geral da Sessão

- **ID:** TESTE-002
- **Aplicação:** ServeRest
- **Testador:** Julliely de Sousa Silva
- **Data:** 12/09/2025
- **Início:** 08/09/2025
- **Fim:** 12/09/2025

APRESENTAÇÃO

O presente planejamento de testes foi desenvolvido com base no estudo e aplicação de técnicas relacionadas à qualidade de software, com foco em **APIs REST**. O objetivo central é garantir que a API **ServeRest** ( [ServeRest](#)) funcione corretamente, atendendo às regras de negócio estabelecidas nas User Stories fornecidas, bem como às especificações definidas no Swagger.

Este documento organiza e detalha todas as etapas do processo de testes, incluindo análise das funcionalidades, definição de cenários de teste, priorização, mapeamento de riscos e cobertura de testes, contribuindo para a entrega de um produto confiável, seguro e com alto padrão de qualidade.

A execução foi realizada em uma **instância EC2 (AWS)**, garantindo maior escalabilidade e representatividade do ambiente de execução real.

Além disso, a automação foi conduzida utilizando o **Robot Framework**, substituindo a abordagem inicial com Postman. Dessa forma, os testes podem ser executados de forma repetitiva, integrada e escalável ao longo do ciclo de vida da aplicação.

OBJETIVO

O objetivo deste plano de testes é:

- Garantir a qualidade funcional da API ServeRest, verificando se as funcionalidades de **cadastro de usuários, autenticação e gerenciamento de produtos** atendem às regras de negócio especificadas.
- Identificar e registrar inconsistências ou erros na aplicação, abrangendo tanto **cenários positivos quanto alternativos**.
- Planejar, organizar e priorizar a execução dos testes, fornecendo **evidências claras e rastreabilidade** entre requisitos, testes e resultados.
- Subsidiar a automação de testes, fornecendo scripts e validações no **Robot Framework**, permitindo execução repetitiva, integração contínua e maior confiabilidade ao longo do desenvolvimento.

ESCOPO

O escopo deste planejamento de testes inclui:

1. APIs e funcionalidades a serem testadas:

- Cadastro de usuários (CRUD) conforme **US 001**.
- Autenticação de usuários e geração de token Bearer conforme **US 002**.
- Cadastro, atualização, listagem e exclusão de produtos conforme **US 003**.

2. Tipos de testes planejados:

- Testes funcionais das APIs (verificação dos endpoints, verbos HTTP e regras de negócio).
- Testes de validação de dados, incluindo regras de e-mail, senha e restrições de duplicidade.
- Testes de segurança relacionados à autenticação e controle de acesso.

3. Limitações do escopo:

- Testes relacionados a APIs externas ou dependentes (ex.: carrinhos de compras) serão considerados somente nas interações críticas.
- Cenários complexos de performance ou carga não estão contemplados neste planejamento inicial.

ANÁLISE

A análise da API **ServeRest** tem como objetivo fornecer uma **visão estratégica** do produto, servindo como ponte entre as funcionalidades da aplicação e os riscos que orientam a priorização dos testes. Ela considera tanto os **aspectos de produto** quanto as **regras de negócio** definidas nas User Stories (US 001, US 002 e US 003) e no Swagger da API.

Aspectos estratégicos identificados:

1. Visão do Produto

- A API ServeRest possibilita que vendedores se cadastrem, autentiquem e gerenciem produtos dentro de um marketplace.
- Funcionalidades críticas incluem cadastro de usuários, autenticação segura e gerenciamento de produtos (CRUD).
- O produto deve suportar corretamente as operações típicas de um marketplace, garantindo integridade e consistência de dados.

2. Regras de Negócio Relevantes

- Campos obrigatórios e validações de entrada (e-mails válidos, restrições de provedores, tamanho de senha).
- Proteção contra duplicidade (usuários e produtos).
- Dependências entre APIs (ex.: produtos não podem ser excluídos se estiverem em carrinhos).
- Geração de tokens de autenticação e restrições de acesso para usuários não autenticados.

3. Riscos Estratégicos que Orientam a Prioridade dos Testes

- **Segurança e autenticação:** falhas podem permitir acesso não autorizado.
- **Integridade de dados:** duplicidade ou exclusão indevida de registros impacta diretamente a operação do marketplace.
- **Confiabilidade das APIs:** endpoints indisponíveis ou mal implementados podem impedir operações críticas de venda.
- **Conformidade com regras de negócio:** ausência de validação de e-mails ou senhas impacta a experiência do usuário e integridade do sistema.

4. Conclusão Estratégica da Análise

- A análise permitiu identificar **prioridades para os testes**, concentrando esforços em áreas críticas para a operação do marketplace: autenticação, cadastro de usuários e gerenciamento de produtos.

- Essa visão estratégica sustenta a definição de **cenários de teste**, **matriz de risco** e **cobertura de testes**, garantindo que os testes executados agreguem valor real ao produto e reduzam potenciais falhas em produção.

TÉCNICAS APLICADAS

Para garantir a efetividade dos testes, foram aplicadas as seguintes técnicas de planejamento de testes:

Partição de Equivalência

- Utilizada para validação de campos de cadastro (ex.: e-mails válidos e inválidos, tamanho da senha).
- Reduz o número de casos de teste mantendo cobertura de cenários relevantes.

Análise de Valor Limite

- Testes realizados nas fronteiras de tamanho de senha, quantidade de produtos e IDs de usuários.
- Garante que o sistema lida corretamente com valores mínimos, máximos e próximos às fronteiras.

Cenários Positivos e Negativos

- **Positivos:** operações de CRUD bem-sucedidas.
- **Negativos:** tentativas de duplicidade, autenticação inválida, exclusão de recursos dependentes.

Mapeamento de Dependências

- Avaliação de interações entre usuários, produtos e carrinhos.
- Identificação de possíveis impactos em cascata devido a operações em uma API dependente.

Rastreabilidade de Requisitos

- Cada teste planejado está vinculado a uma **User Story** e ao endpoint correspondente no **Swagger**.
- Facilita a documentação de evidências e a abertura de issues.

Planejamento para Automação

- Identificação de testes candidatos à automação utilizando o **Robot Framework**, aproveitando sua capacidade de estruturar casos de teste

de forma legível e reutilizável.

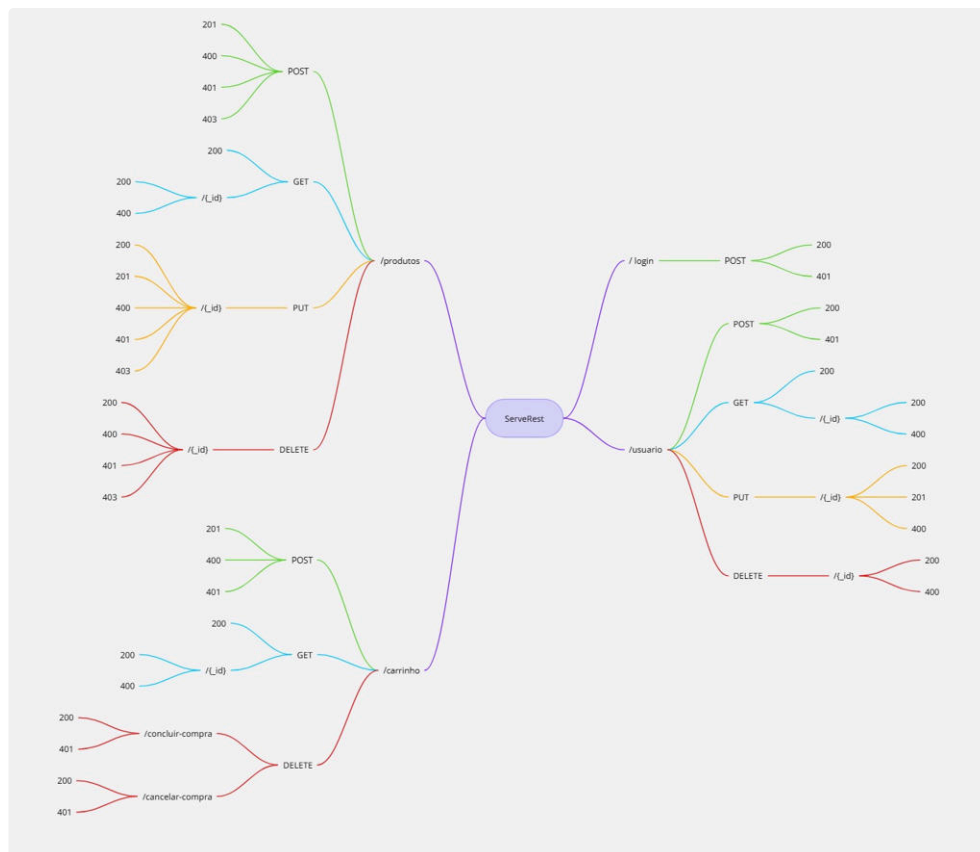
- Foco em testes repetitivos, críticos e que envolvam validação de regras de negócio.
- Execução em **instância EC2 (AWS)**, permitindo maior escalabilidade e integração ao ciclo de desenvolvimento

FERRAMENTAS UTILIZADAS

Para a execução e automação dos testes da API **ServeRest**, foram utilizadas as seguintes ferramentas e recursos:

- **Robot Framework**
 - Framework de automação de testes orientado a palavras-chave (keyword-driven).
 - Utilizado para estruturar e automatizar os cenários de teste da API.
- **RequestsLibrary**
 - Biblioteca do Robot Framework voltada para chamadas HTTP.
 - Usada para interagir diretamente com os endpoints da API.
- **FakerLibrary**
 - Biblioteca utilizada para geração de dados dinâmicos (e-mails, nomes, senhas, IDs).
 - Reduz a necessidade de massa de dados fixa e previne falhas por duplicidade.
- **Instância EC2 (AWS)**
 - Ambiente de execução configurado para rodar os testes de forma remota.
 - Garante maior escalabilidade, disponibilidade e proximidade de um ambiente real de produção.
- **Swagger (ServeRest)**
 - Documentação oficial da API.
 - Utilizada como referência para definição dos cenários de teste e validação das respostas esperadas.

MAPA MENTAL DA APLICAÇÃO



CENÁRIOS DE TESTES PLANEJADOS

1. Usuários (US 001)

Cenários Positivos:

- **Cadastro de novo usuário válido**

Verifica se um usuário pode ser cadastrado com nome, e-mail válido, senha dentro do limite e flag de administrador. *(Implementado e passou nos testes)*

- **Atualização de usuário existente**

Verifica se as alterações completas ou parciais nos dados de um usuário existente são aplicadas corretamente. *(Implementado e passou nos testes)*

- **Listagem de usuários cadastrados**

Confirma se a API retorna corretamente todos os usuários e também usuários por ID válido. *(Implementado e passou nos testes)*

- **Exclusão de usuário existente**

Verifica se um usuário pode ser removido da base de dados sem afetar integridade do sistema. *(Implementado e passou nos testes)*

Cenários Alternativos / Negativos:

- **Cadastro de usuário com e-mail duplicado**

Testa se a API impede criação de usuário com e-mail já cadastrado.
(Implementado e passou nos testes)

- **Cadastro de usuário com e-mail inválido (hotmail ou formato incorreto)**

Verifica se o sistema bloqueia provedores não permitidos e e-mails inválidos. (Implementado, mas apresentou falhas parciais)

- **Cadastro de usuário com senha fora do limite (menos de 5 ou mais de 10 caracteres)**

Avalia se o sistema aplica corretamente a validação de senha.
(Implementado, mas apresentou falhas parciais)

- **Atualização de usuário inexistente via PUT**

Verifica se um novo usuário é criado automaticamente ao tentar atualizar um ID inexistente. (Implementado e passou nos testes)

- **Exclusão de usuário inexistente ou já excluído**

Verifica se o sistema retorna 200 sem excluir dados quando o usuário não existe. (Implementado e passou nos testes).

2. Login (US 002)

Cenários Positivos:

- **Autenticação de usuário válido**

Usuário cadastrado realiza login corretamente e recebe token Bearer válido. (Implementado e passou nos testes)

Cenários Alternativos / Negativos:

- **Autenticação de usuário não cadastrado**

Verifica se a API retorna erro 401 (Unauthorized). (Implementado e passou nos testes)

- **Autenticação com senha incorreta**

Garante que o sistema não autentica e retorna status 401.
(Implementado e passou nos testes)

- **Autenticação com campos vazios**

Garante que a API retorna erro 400 quando os campos obrigatórios não são informados. (Implementado e passou nos testes)

- **Validade do token Bearer**

Confirma que o token gerado expira corretamente após 10 minutos.
(Ainda não implementado nos testes)

3. Produtos (US 003)

Cenários Positivos:

- **Cadastro de novo produto válido**

Usuário autenticado cadastra produto com dados válidos.
(Implementado e passou nos testes)

- **Atualização de produto existente**

Verifica se produto existente é atualizado corretamente. *(Implementado e passou nos testes)*

- **Listagem de produtos**

Confirma que todos os produtos cadastrados são retornados corretamente, bem como listagem por ID válido. *(Implementado e passou nos testes)*

- **Exclusão de produto sem dependência em carrinho**

Produto é removido sem afetar outras APIs. *(Implementado e passou nos testes)*

Cenários Alternativos / Negativos:

- **Cadastro de produto com nome duplicado**

Testa se o sistema bloqueia produtos com nomes já existentes.
(Implementado e passou nos testes)

- **Exclusão de produto presente em carrinho**

Avalia se a API impede exclusão de produtos vinculados a carrinhos.
(Implementado, mas apresentou falhas nos testes)

- **Atualização de produto inexistente via PUT**

Verifica se um novo produto é criado automaticamente ao tentar atualizar um ID inexistente. *(Implementado e passou nos testes)*

- **Ações de produtos por usuário não autenticado**

Testa se a API retorna erro 401 para usuários não logados.
(Implementado e passou nos testes)

- **Ações de produtos por usuário não administrador**

Verifica se usuários sem perfil de administrador recebem erro 403 ao tentar manipular produtos. *(Implementado e passou nos testes)*

Priorização

1. Alta:

- Cadastro e autenticação de usuários.
- Cadastro e manutenção de produtos.

2. Média:

- Atualizações e listagens de usuários e produtos.

3. Baixa:

- Cenários alternativos menos críticos (e-mails inválidos, senhas fora do limite).
- Teste de expiração do token Bearer (não implementado).
- Exclusão de produtos vinculados a carrinhos (falho).

MATRIZ DE RISCO

[Elaborar uma matriz de risco e definir em questão de probabilidade e impacto, sua posição em uma tabela.

Probabilidade \ Impacto	Sem impacto	Leve	Médio	Grave	Gravíssimo
Quase certo	Cadastro com e-mail inválido (bloqueio correto)		Falha em validação de senha (curta/longa)	Usuário não autenticado acessa rota de produtos	Perda de dados de usuário/ produto crítico
Alta		Cadastro de usuário com e-mail duplicado (bloqueado corretamente)	Falha em autenticação de login	Exclusão de produto em carrinho (falha real)	Falha na atualização/criação de usuário crítico

				observada)	
Média			Atualização de usuário inexistente via PUT (gera novo)		Falha na geração/ expiração de token
Baixa		Listagem de usuários/ produtos com dados inconsistentes	Atualização de usuário existente		
Raro			Falha em endpoints pouco usados		Falha geral de API durante pico de uso

Legenda:

Cinza - Baixo risco

Verde - Risco moderado

Amarelo - Risco elevado

Vermelho - Risco extremo

COBERTURA DE TESTES

Foram utilizadas dois métodos de análise de cobertura de testes, eles são:

1. **Status Code Coverage (Cobertura de Código de Status):** Este método de cobertura verifica se o conjunto de testes executa todos os códigos de resposta HTTP possíveis para cada endpoint. Seu conjunto de testes

aborda cenários de sucesso (`200 OK` , `201 Created`), falhas do cliente (`400 Bad Request` , `401 Unauthorized` , `403 Forbidden`) e casos de "não encontrado" (`404 Not Found`). Essa abordagem garante que a API responde corretamente a diferentes tipos de requisições, sejam elas bem-sucedidas ou mal-sucedidas.

2. **Parameter Value Coverage (Cobertura de Valor de Parâmetro):** Este método foca na validação dos dados de entrada (input) que a API recebe, incluindo a verificação de valores válidos, inválidos e de limite. Nos seus testes, foram utilizados:
- **Valores Válidos:** Dados que estão em conformidade com o formato e as restrições esperadas (ex: login com credenciais corretas).
 - **Valores Inválidos:** Dados que não seguem as regras de validação da API (ex: e-mail ou senha incorretos, campos vazios, ID inexistente).
 - **Valores de Limite (Boundary):** Dados que testam os limites da validação (ex: senhas com 5 e 10 caracteres, que são os limites aceitáveis).

Usuários

- **Status Code Coverage:** ~77%
(já cobrimos 201, 200, 400 em alguns fluxos; falta 401 e alguns erros de campos inválidos).
- **Parameter Value Coverage:** ~68%
(foram testados casos de dados válidos, duplicados e inexistentes; ainda faltam nulos e filtros).

Produtos

- **Status Code Coverage:** ~75%
(cobertura boa para criação, listagem e atualização, mas ainda faltam 400 para inválidos e 401 para não autenticado).
 - **Parameter Value Coverage:** ~65%
(temos testes para duplicados e válidos; faltam campos obrigatórios e filtros de pesquisa).
-

Login

- **Status Code Coverage:** 100%
(sucesso, senha incorreta e usuário inexistente já cobertos).
- **Parameter Value Coverage:** 100%
(todos os cenários críticos de entrada já testados).

Cobertura Geral

- **Status Code Coverage (média geral): ~81%**
- **Parameter Value Coverage (média geral): ~73%**

A tabela a seguir detalha a aplicação desses métodos para cada endpoint testado na sua coleção.

Endpoint	Método	Status Codes Testados	Parâmetros Testados	Descrição da Cobertura
POST Login	POST	200, 400, 401	Válidos: E-mail e senha corretos. Inválidos: E-mail inválido, senha incorreta, campos vazios.	Alta cobertura, testando autenticação com sucesso e diversas falhas de autenticação.
POST Criar usuário	POST	201, 400	Válidos: Nome, e-mail e senha corretos. Inválidos: E-mail duplicado, campos vazios, e-mails de domínios	Cobertura abrangente para criação de usuários, incluindo valores válidos, inválidos e limites de senha.

			específicos. Limites: Senhas fora dos limites de 5 a 10 caracteres.	
GET Listar usuários	GET	200	Válidos: Listar todos os usuários, filtrar por nome e e-mail válidos. Inválidos: Filtro por nome inexistente, filtro com parâmetro inválido.	Boa cobertura para listagem de usuários com e sem filtros.
GET Buscar por ID	GET	200, 400	Válidos: ID existente. Inválidos: ID inexistente, ID com tamanho inválido.	Cobertura adequada, com cenários de sucesso e falha.
PUT Editar usuário	PUT	200, 201, 400	Válidos: Atualização completa e parcial de usuário existente. Inválidos: ID inexistente (que cria um novo usuário).	Boa cobertura para atualizações, testando dados válidos, limites e o comportamento com ID inexistente.

			Limites: Senhas com 5 e 10 caracteres.	
DELETE usuário	DELETE	200, 400	Válidos: ID existente. Inválidos: ID inexistente, ID com caracteres inválidos.	Cobertura básica para exclusão de usuário, testando cenários de sucesso e falha.
GET Produtos	GET	200, 400, 401, 403	Válidos: Listar produtos com token válido, filtrar por nome válido. Inválidos: Sem autenticação, filtros inválidos.	Boa cobertura, incluindo cenários de autenticação e filtros.
GET Produtos por ID	GET	200, 400	Válidos: ID de produto existente. Inválidos: ID de produto inválido ou inexistente.	Cobertura para busca de produtos por ID, testando cenários de sucesso e falha.
POST Produtos	POST	201, 400, 401, 403	Válidos: Dados de produto válidos.	Cobertura abrangente para criação de produtos,

			Inválidos: Nome muito longo, preço negativo, quantidade negativa.	incluindo validação de dados de entrada e seus limites.
DELETE Produtos por ID	DELETE	200, 401, 403, 400	Válidos: Exclusão com token de administrador válido. Inválidos: Sem autenticação, usuário sem permissão de administrador.	Cobertura para exclusão de produtos, testando cenários de sucesso e falhas de autenticação /autorização.
PUT Produtos por ID	PUT	200, 201, 400, 401, 403	Válidos: Atualização de produto com dados válidos. Inválidos: ID inexistente, sem autenticação, dados inválidos no corpo.	Alta cobertura para atualizações, testando autenticação, dados inválidos e o comportamento com ID inexistente.

Testes Já Automatizados

Login (US 002):

- Autenticação válida (200).

- Autenticação inválida → e-mail incorreto, senha incorreta, campos obrigatórios vazios (401 e 400).

Cadastro de Usuários (US 001):

- Usuário válido (201).
- E-mail duplicado (400).
- Campos obrigatórios vazios (400).
- E-mail inválido/domínio restrito (400).
- Senha inválida (curta <5 ou longa >10) (400).

Usuários – Operações adicionais:

- Listar todos os usuários (200).
- Buscar usuário por ID válido (200) e inválido/inexistente (400).
- Atualização completa e parcial de usuário (200).
- Atualização com e-mail duplicado (400).
- PUT com ID inexistente criando novo usuário (201).
- Exclusão de usuário válido (200).
- Exclusão de usuário inexistente ou já excluído (200).

Produtos (US 003):

- Listar produtos autenticado (200).
- Tentativa de listar sem autenticação (401).
- Buscar produto por ID válido (200) e inexistente (400).
- Cadastro de produto válido (201).
- Cadastro de produto já existente (400).
- Cadastro sem token (401) ou com usuário não admin (403).
- Atualização de produto existente com dados válidos (200).
- PUT produto inexistente cria novo (201).
- PUT produto com nome duplicado (400).
- PUT com usuário não admin (403).
- Exclusão de produto válido (200).
- Exclusão sem autenticação (401).
- Exclusão de produto em carrinho (400).